

1. Reinforcement Learning for Delivery Route Optimization

About the Question

This question is about using Reinforcement Learning (RL) to find efficient delivery routes in a logistics system. A delivery vehicle may face traffic, different road distances, road closures, and multiple delivery locations.

How RL is Used

The RL agent observes the current delivery situation and chooses a route. If the selected route results in a faster delivery or lower cost, it receives a positive reward. If the route causes delay or extra travel, it can receive a penalty. By trying routes repeatedly, the system learns which routes work better.

RL Components

Agent: Delivery vehicle or route management system.

Environment: Roads, customers, traffic, weather, and delivery locations.

State: Current vehicle location, traffic, remaining deliveries, and time.

Action: Select the next route or delivery location.

Reward: Positive reward for fast/on-time delivery and penalty for delays.

Policy: Strategy used to select the best route.

Continuous Learning

Traffic conditions change throughout the day. By continuously learning from completed deliveries, the agent can recognize which roads are normally congested and adapt when conditions change. This can reduce delivery time, fuel use, and cost.

Simple Executable Python Code

```
import random
```

```
routes = ["Route A", "Route B", "Route C"]
```

```
rewards = [5, 10, 3]
```

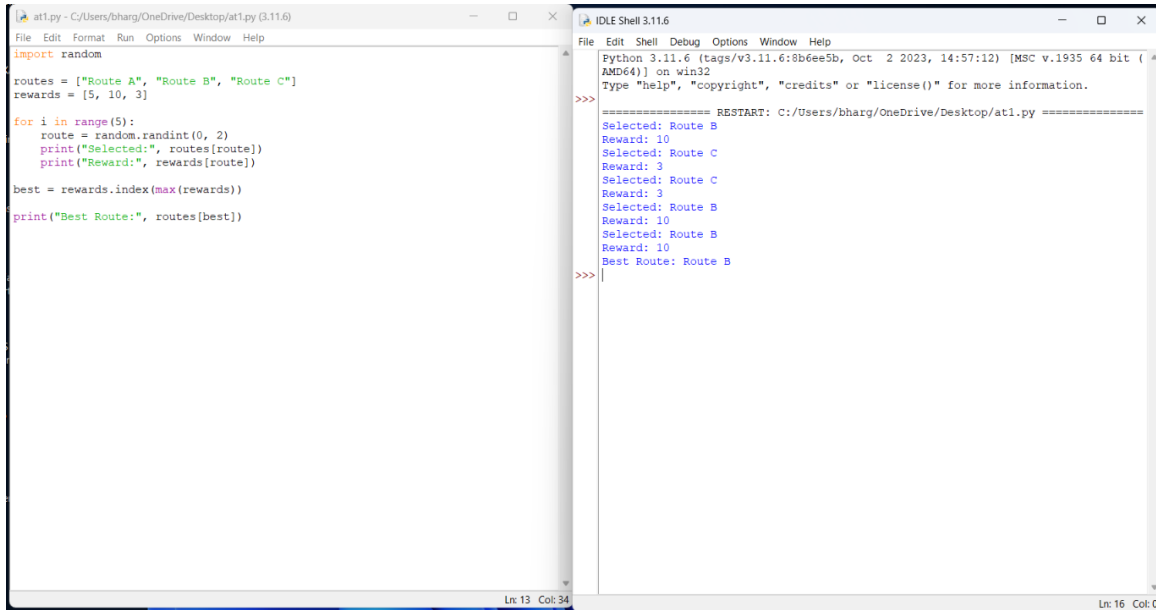
```
for i in range(5):
```

```
    route = random.randint(0, 2)
```

```
    print("Selected:", routes[route])
```

```
    print("Reward:", rewards[route])
```

```
best = rewards.index(max(rewards))
print("Best Route:", routes[best])
```



The image shows a screenshot of a Python IDE with two windows. The left window, titled 'at1.py - C:/Users/bharg/OneDrive/Desktop/at1.py (3.11.6)', contains the following code:

```
import random
routes = ["Route A", "Route B", "Route C"]
rewards = [5, 10, 3]

for i in range(5):
    route = random.randint(0, 2)
    print("Selected:", routes[route])
    print("Reward:", rewards[route])

best = rewards.index(max(rewards))
print("Best Route:", routes[best])
```

The right window, titled 'IDLE Shell 3.11.6', shows the output of the script:

```
Python 3.11.6 (tags/v3.11.6:8b6ee5b, Oct 2 2023, 14:57:12) [MSC v.1935 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/bharg/OneDrive/Desktop/at1.py =====
Selected: Route B
Reward: 10
Selected: Route C
Reward: 3
Selected: Route C
Reward: 3
Selected: Route B
Reward: 10
Selected: Route B
Reward: 10
Best Route: Route B
>>>
```

2. Reinforcement Learning for Fraud Detection in Fintech

About the Question

This question is about applying RL to detect suspicious or fraudulent financial transactions.

How RL is Used

The agent observes transaction information such as amount, time, location, and user behaviour. It can approve, reject, or send a transaction for review. Correct decisions receive positive rewards, while incorrect decisions receive penalties.

RL Components

Agent: Fraud detection system.

Environment: Banking or fintech transaction system.

State: Transaction amount, location, time, device, and previous user behaviour.

Action: Approve, reject, or review a transaction.

Reward: Positive reward for correct detection and penalty for wrong decisions.

Policy: Strategy for deciding how to handle a transaction.

Exploration Strategies

Fraud patterns can change. Exploration allows the agent to try different actions for uncertain cases instead of always using the same known decision. For example, an unusual transaction can be sent for manual review. Feedback from such cases can help the system discover new fraud patterns and improve detection accuracy.

Simple Executable Python Code

```

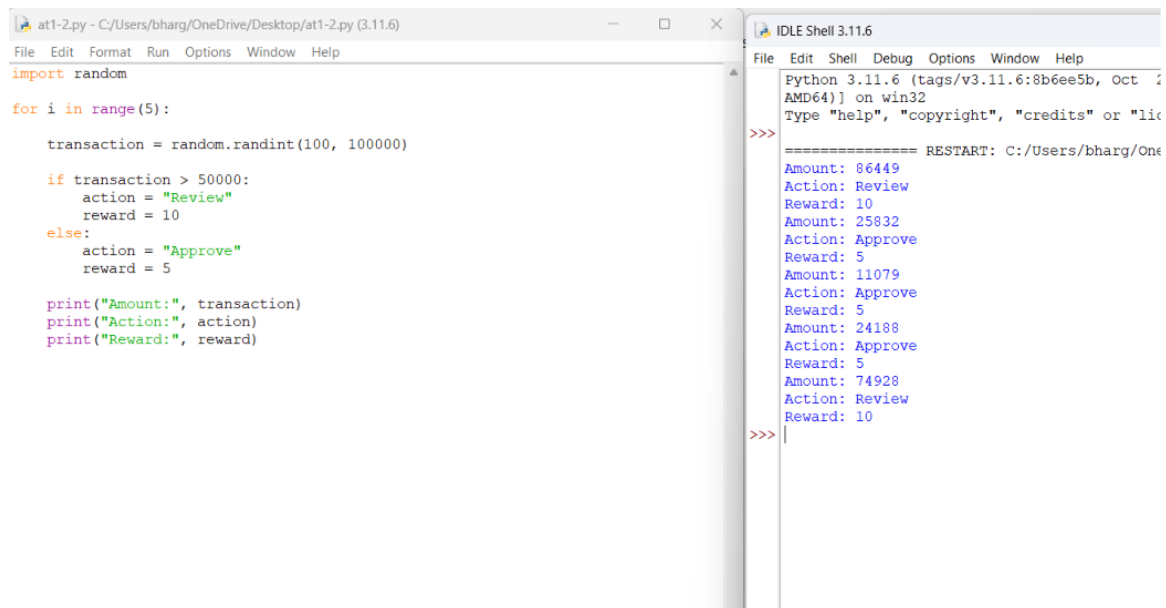
import random

for i in range(5):
    transaction = random.randint(100, 100000)

    if transaction > 50000:
        action = "Review"
        reward = 10
    else:
        action = "Approve"
        reward = 5

    print("Amount:", transaction)
    print("Action:", action)
    print("Reward:", reward)
    print()

```



The screenshot shows a Python IDE with two panes. The left pane displays the source code of a script named 'at1-2.py'. The right pane shows the output of the script in the IDLE Shell.

Source Code (Left Pane):

```

import random

for i in range(5):
    transaction = random.randint(100, 100000)

    if transaction > 50000:
        action = "Review"
        reward = 10
    else:
        action = "Approve"
        reward = 5

    print("Amount:", transaction)
    print("Action:", action)
    print("Reward:", reward)
    print()

```

Output (Right Pane):

```

Python 3.11.6 (tags/v3.11.6:8b6ee5b, Oct 2
AMD64)] on win32
Type "help", "copyright", "credits" or "lic
>>>
===== RESTART: C:/Users/bharg/One
Amount: 86449
Action: Review
Reward: 10
Amount: 25832
Action: Approve
Reward: 5
Amount: 11079
Action: Approve
Reward: 5
Amount: 24188
Action: Approve
Reward: 5
Amount: 74928
Action: Review
Reward: 10
>>>

```

3. Reinforcement Learning for Content Recommendation

About the Question

This question is about using RL in a streaming platform to recommend movies, videos, songs, or shows according to user interests.

How RL is Used

The agent observes the user's history and recommends content. The user's response provides feedback. Watching or liking recommended content can give a positive reward, while immediately skipping it can give a negative reward.

RL Components

Agent: Recommendation system.

Environment: Streaming platform and users.

State: Watch history, likes, searches, watch time, and previous recommendations.

Action: Recommend a movie, video, song, or category.

Reward: Positive reward when the user watches or likes content; penalty when content is skipped.

Policy: Strategy for selecting content that is likely to interest the user.

Delayed Rewards, Feedback, and Changing Preferences

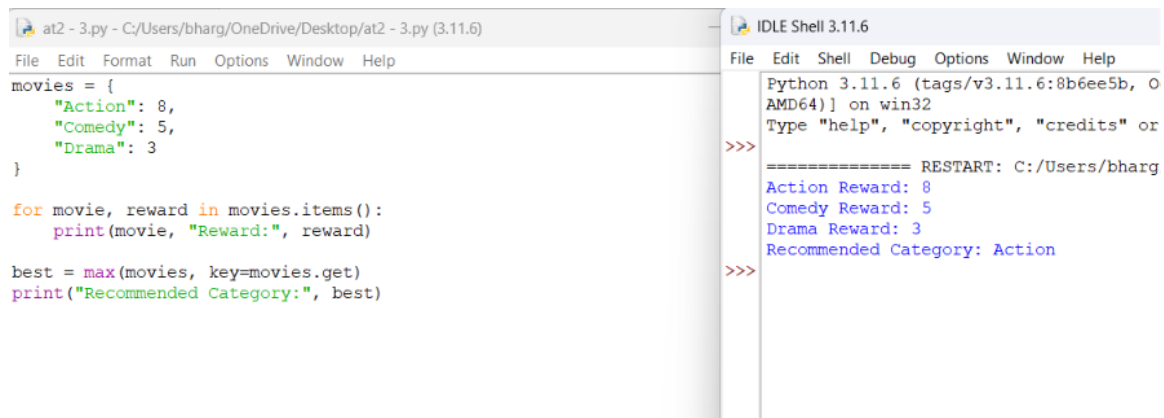
A user may save a movie today and watch it several days later. The reward therefore arrives later, which is called a delayed reward. Likes, dislikes, watch time, ratings, and skips provide feedback. User preferences can also change over time, so the agent must continuously update its recommendations.

Simple Executable Python Code

```
movies = {
    "Action": 8,
    "Comedy": 5,
    "Drama": 3
}

for movie, reward in movies.items():
    print(movie, "Reward:", reward)

best = max(movies, key=movies.get)
print("Recommended Category:", best)
```

A screenshot of a Python IDE window titled 'at2 - 3.py - C:/Users/bharg/OneDrive/Desktop/at2 - 3.py (3.11.6)'. The code editor on the left contains the same Python code as shown in the previous block. The output window on the right, titled 'IDLE Shell 3.11.6', shows the execution results: 'Python 3.11.6 (tags/v3.11.6:8b6ee5b, O AMD64)] on win32', 'Type "help", "copyright", "credits" or >>>', '==== RESTART: C:/Users/bharg', 'Action Reward: 8', 'Comedy Reward: 5', 'Drama Reward: 3', and 'Recommended Category: Action'.

4. Reinforcement Learning for Predictive Maintenance

About the Question

This question is about using RL to decide when industrial machines should be inspected, repaired, or allowed to continue operating.

How RL is Used

Sensors provide information such as temperature, vibration, pressure, and operating time. The agent uses this information to choose actions such as continue operation, inspect, or perform maintenance. Preventing a breakdown gives a positive reward, while a machine failure can produce a large penalty.

RL Components

Agent: Maintenance decision system.

Environment: Industrial machine and factory.

State: Temperature, vibration, pressure, machine health, and operating time.

Action: Continue, inspect, repair, or stop.

Reward: Positive reward for preventing failure and penalty for breakdown or unnecessary maintenance.

Policy: Strategy for deciding when maintenance should be performed.

RL vs Traditional Rule-Based Approach

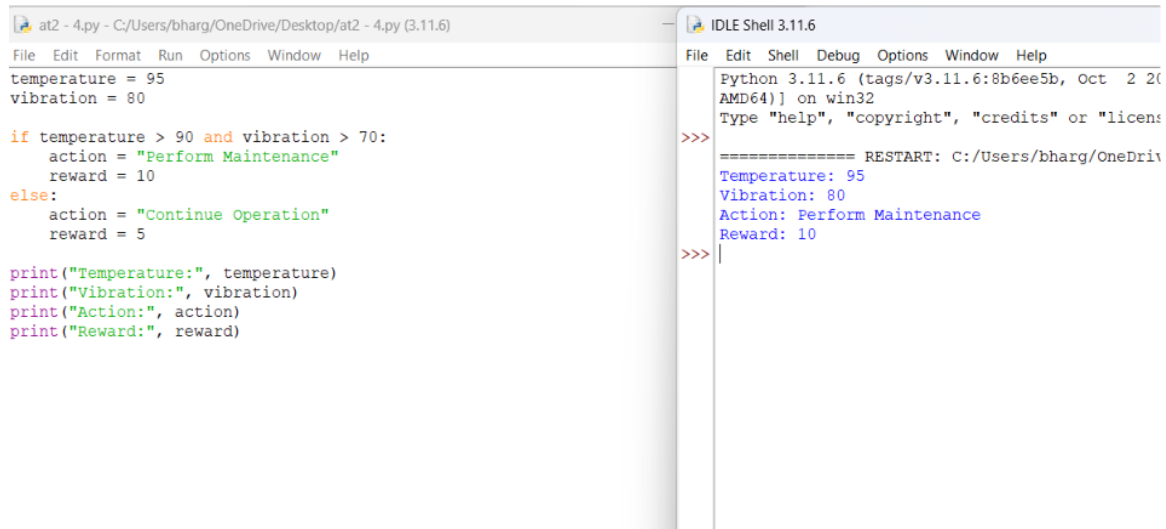
A traditional system uses fixed rules, such as stopping a machine when temperature exceeds a particular value. RL can learn from previous machine behaviour and consider several conditions together. However, industrial RL has risks: incorrect decisions, poor sensor data, insufficient training, and unexpected machine conditions can affect reliability. Safety checks and human supervision are important for critical systems.

Simple Executable Python Code

```
temperature = 95
vibration = 80

if temperature > 90 and vibration > 70:
    action = "Perform Maintenance"
    reward = 10
else:
    action = "Continue Operation"
    reward = 5

print("Temperature:", temperature)
print("Vibration:", vibration)
print("Action:", action)
print("Reward:", reward)
```



```
at2 - 4.py - C:/Users/bharg/OneDrive/Desktop/at2 - 4.py (3.11.6)
File Edit Format Run Options Window Help
temperature = 95
vibration = 80

if temperature > 90 and vibration > 70:
    action = "Perform Maintenance"
    reward = 10
else:
    action = "Continue Operation"
    reward = 5

print("Temperature:", temperature)
print("Vibration:", vibration)
print("Action:", action)
print("Reward:", reward)
```

```
IDLE Shell 3.11.6
File Edit Shell Debug Options Window Help
Python 3.11.6 (tags/v3.11.6:8b6ee5b, Oct 2 2023, [AMD64]) on win32
Type "help", "copyright", "credits" or "license()" for more
>>>
===== RESTART: C:/Users/bharg/OneDrive/Desktop/at2 - 4.py (3.11.6) =====
Temperature: 95
Vibration: 80
Action: Perform Maintenance
Reward: 10
>>>
```

5. Reinforcement Learning for Transportation Optimization

About the Question

This question is about using RL to improve transportation systems such as bus scheduling and route allocation.

How RL is Used

The agent observes passenger demand, bus locations, traffic, and available vehicles. It can change schedules, allocate an additional bus, or select another route. The objective is to reduce passenger waiting time and travel time.

RL Components

Agent: Transportation management system.

Environment: Roads, buses, passengers, and traffic.

State: Passenger demand, traffic, bus locations, current time, and available buses.

Action: Send another bus, change route, or change schedule.

Reward: Positive reward for reducing waiting/travel time and penalties for delays.

Policy: Strategy for allocating buses and routes efficiently.

Real-Time Data

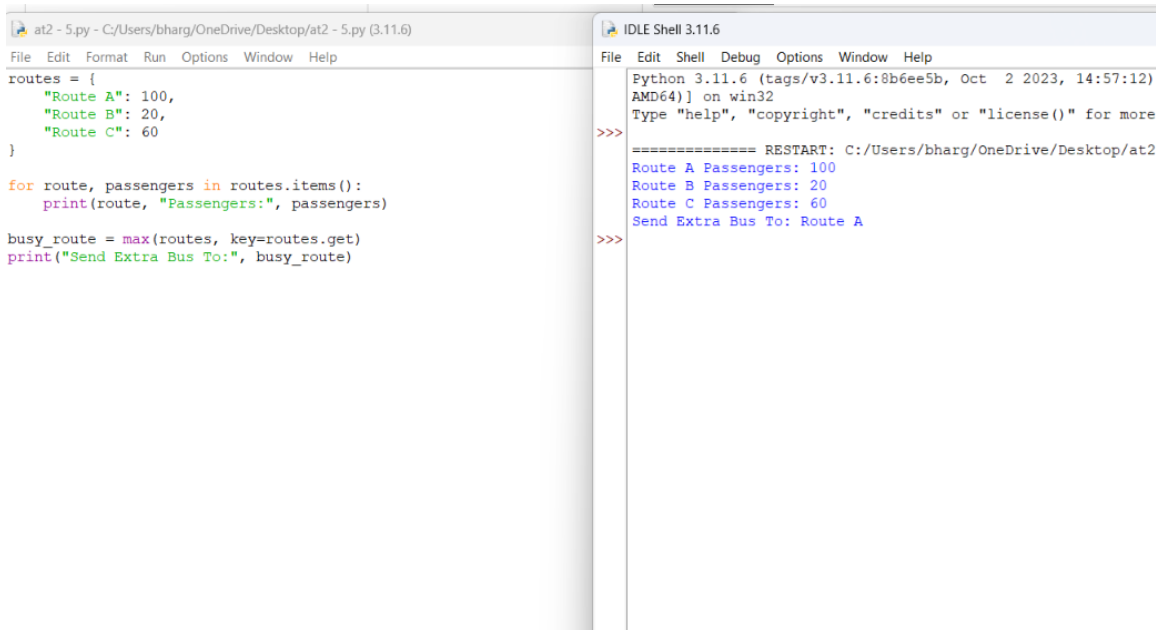
GPS, traffic sensors, passenger demand, accidents, and road-closure information can update the state in real time. The agent can respond to these changes and choose a better route or bus allocation.

Simple Executable Python Code

```
routes = {
    "Route A": 100,
    "Route B": 20,
    "Route C": 60
}

for route, passengers in routes.items():
    print(route, "Passengers:", passengers)

busy_route = max(routes, key=routes.get)
print("Send Extra Bus To:", busy_route)
```

The image shows a screenshot of a Python script editor and its execution. On the left, a file named 'at2 - 5.py' is open, showing a dictionary of routes and a loop that prints the passengers for each route. The script identifies 'Route A' as the busiest. On the right, the IDLE Shell shows the execution output, which matches the script's logic and identifies 'Route A' as the route to which an extra bus should be sent.

```
at2 - 5.py - C:/Users/bharg/OneDrive/Desktop/at2 - 5.py (3.11.6)
File Edit Format Run Options Window Help
routes = {
    "Route A": 100,
    "Route B": 20,
    "Route C": 60
}

for route, passengers in routes.items():
    print(route, "Passengers:", passengers)

busy_route = max(routes, key=routes.get)
print("Send Extra Bus To:", busy_route)
```

```
IDLE Shell 3.11.6
File Edit Shell Debug Options Window Help
Python 3.11.6 (tags/v3.11.6:8b6ee5b, Oct 2 2023, 14:57:12)
AMD64) on win32
Type "help", "copyright", "credits" or "license()" for more
>>>
===== RESTART: C:/Users/bharg/OneDrive/Desktop/at2
Route A Passengers: 100
Route B Passengers: 20
Route C Passengers: 60
Send Extra Bus To: Route A
>>>
```

6. Reinforcement Learning for Dynamic Pricing in E-Commerce

About the Question

This question is about using RL to change product prices according to customer demand and market conditions.

How RL is Used

The agent observes information such as current price, demand, inventory, competitor prices, and time. It then chooses a price. The reward can be based on revenue or profit. By repeating this process, the agent learns which prices work better under different conditions.

RL Components

Agent: Dynamic pricing system.

Environment: E-commerce market and customers.

State: Product demand, current price, inventory, competitor prices, and time.

Action: Increase price, decrease price, or keep the same price.

Reward: Revenue/profit, with penalties for poor sales or undesirable outcomes.

Policy: Strategy for selecting a suitable price.

Exploration vs Exploitation

Exploration means trying a new price to learn how customers respond. Exploitation means using a price that is already known to work well. Too much exploration can reduce sales, while too much exploitation may prevent the system from discovering better prices. RL must balance both.

Simple Executable Python Code

```
demand = 80
```

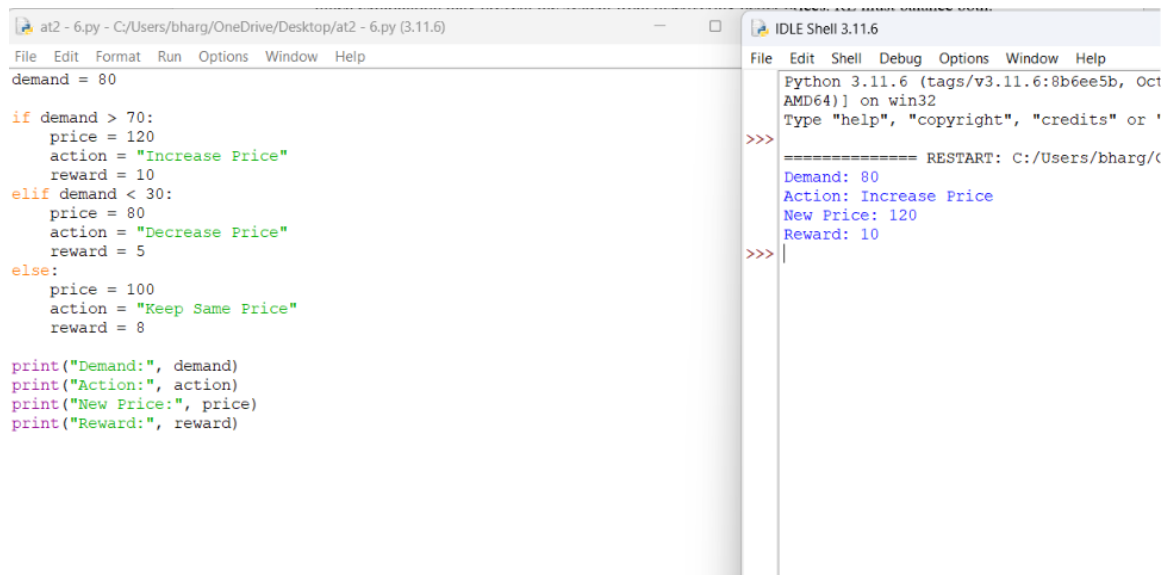
```
if demand > 70:
```

```

    price = 120
    action = "Increase Price"
    reward = 10
elif demand < 30:
    price = 80
    action = "Decrease Price"
    reward = 5
else:
    price = 100
    action = "Keep Same Price"
    reward = 8

print("Demand:", demand)
print("Action:", action)
print("New Price:", price)
print("Reward:", reward)

```



The screenshot shows a Python IDE with two windows. The left window, titled 'at2 - 6.py - C:/Users/bharg/OneDrive/Desktop/at2 - 6.py (3.11.6)', contains the following code:

```

demand = 80

if demand > 70:
    price = 120
    action = "Increase Price"
    reward = 10
elif demand < 30:
    price = 80
    action = "Decrease Price"
    reward = 5
else:
    price = 100
    action = "Keep Same Price"
    reward = 8

print("Demand:", demand)
print("Action:", action)
print("New Price:", price)
print("Reward:", reward)

```

The right window, titled 'IDLE Shell 3.11.6', shows the execution output:

```

Python 3.11.6 (tags/v3.11.6:8b6ee5b, Oct
AMD64) on win32
Type "help", "copyright", "credits" or '
>>>
===== RESTART: C:/Users/bharg/<
Demand: 80
Action: Increase Price
New Price: 120
Reward: 10
>>>

```

7. Reinforcement Learning for Smart City Waste Management

About the Question

This question is about using RL to manage waste collection efficiently in a smart city.

How RL is Used

Smart bins can provide information about how full they are. The RL agent decides which bins or areas a collection vehicle should visit and which route it should take. The goal is to collect full bins while reducing unnecessary travel, fuel consumption, and time.

MDP / RL Components

Agent: Waste collection or route management system.

Environment: City roads, waste bins, collection vehicles, and traffic.

State: Bin fill level, truck location, traffic, and vehicle capacity.

Action: Select the next bin, collection area, or route.

Reward: Positive reward for collecting full bins efficiently and penalties for overflow or unnecessary travel.

Policy: Strategy for choosing the best collection route.

Continuous Updates

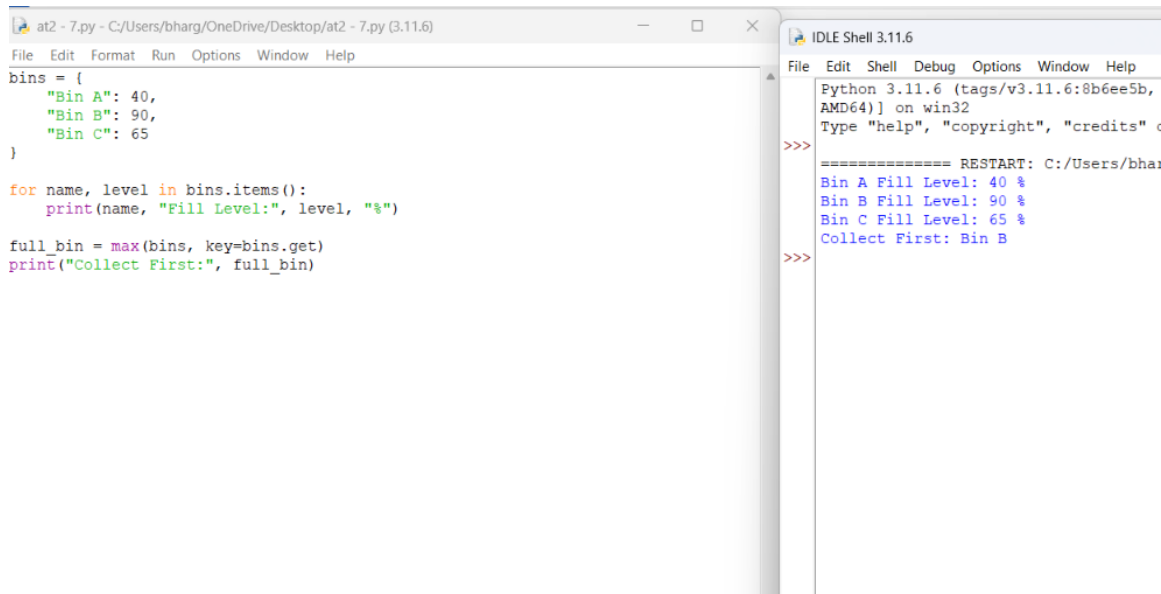
Bin levels and traffic conditions change throughout the day. Continuous updates allow the agent to change collection priorities and routes using new sensor and traffic information. This improves operational efficiency.

Simple Executable Python Code

```
bins = {
    "Bin B": 90,
    "Bin C": 65
}

for name, level in bins.items():
    print(name, "Fill Level:", level, "%")

full_bin = max(bins, key=bins.get)
print("Collect First:", full_bin)
```



```
at2 - 7.py - C:/Users/bharg/OneDrive/Desktop/at2 - 7.py (3.11.6)
File Edit Format Run Options Window Help
bins = {
    "Bin A": 40,
    "Bin B": 90,
    "Bin C": 65
}

for name, level in bins.items():
    print(name, "Fill Level:", level, "%")

full_bin = max(bins, key=bins.get)
print("Collect First:", full_bin)
```

```
IDLE Shell 3.11.6
File Edit Shell Debug Options Window Help
Python 3.11.6 (tags/v3.11.6:8b6ee5b, AMD64) on win32
Type "help", "copyright", "credits" or ">>>"
>>>
===== RESTART: C:/Users/bha
Bin A Fill Level: 40 %
Bin B Fill Level: 90 %
Bin C Fill Level: 65 %
Collect First: Bin B
>>>
```

8. Reinforcement Learning for Network Bandwidth Allocation

About the Question

This question is about using RL to distribute network bandwidth efficiently among users or applications in a telecommunications network.

How RL is Used

The agent observes network traffic, user demand, available bandwidth, and congestion. It decides

how much bandwidth should be allocated to different users or services. Good allocation can improve network performance and reduce congestion.

RL Components

Agent: Network bandwidth controller.

Environment: Telecommunication network, users, and applications.

State: Available bandwidth, network traffic, number of users, and congestion.

Action: Increase, decrease, or redistribute bandwidth.

Reward: Positive reward for high network performance and low congestion; penalties for delays or poor service.

Policy: Strategy for allocating bandwidth efficiently.

Adapting to Dynamic Conditions

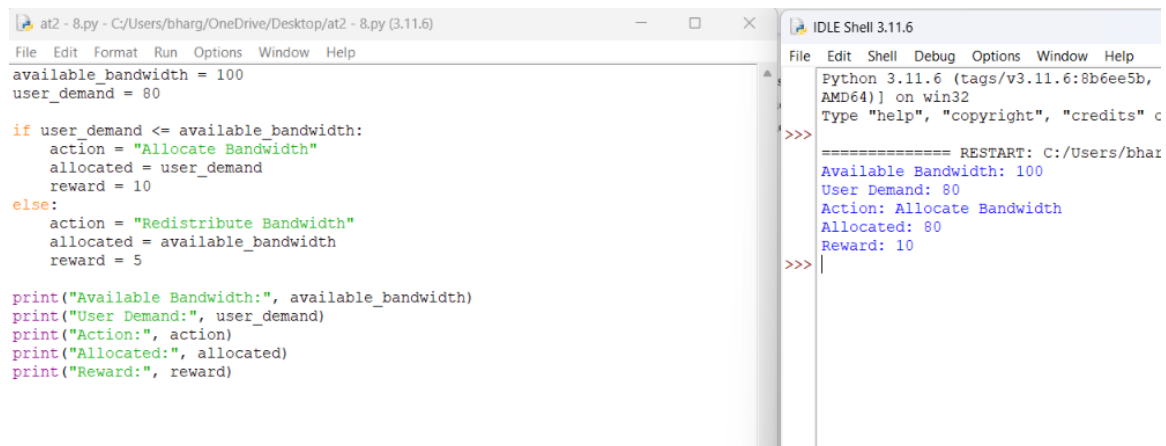
Network demand changes continuously. For example, video streaming may suddenly increase traffic. RL can observe the new state and redistribute bandwidth. Continuous learning allows the system to adapt to changing users, traffic, and service demands.

Simple Executable Python Code

```
available_bandwidth = 100
user_demand = 80

if user_demand <= available_bandwidth:
    action = "Allocate Bandwidth"
    allocated = user_demand
    reward = 10
else:
    action = "Redistribute Bandwidth"
    allocated = available_bandwidth
    reward = 5

print("Available Bandwidth:", available_bandwidth)
print("User Demand:", user_demand)
print("Action:", action)
print("Allocated:", allocated)
print("Reward:", reward)
```



```
at2 - 8.py - C:/Users/bharg/OneDrive/Desktop/at2 - 8.py (3.11.6)
File Edit Format Run Options Window Help
available_bandwidth = 100
user_demand = 80

if user_demand <= available_bandwidth:
    action = "Allocate Bandwidth"
    allocated = user_demand
    reward = 10
else:
    action = "Redistribute Bandwidth"
    allocated = available_bandwidth
    reward = 5

print("Available Bandwidth:", available_bandwidth)
print("User Demand:", user_demand)
print("Action:", action)
print("Allocated:", allocated)
print("Reward:", reward)
```

```
IDLE Shell 3.11.6
File Edit Shell Debug Options Window Help
Python 3.11.6 (tags/v3.11.6:8b6ee5b,
AMD64) on win32
Type "help", "copyright", "credits" c
>>>
===== RESTART: C:/Users/bhar
Available Bandwidth: 100
User Demand: 80
Action: Allocate Bandwidth
Allocated: 80
Reward: 10
>>>
```

9. Reinforcement Learning in Portfolio Management

About the Question

This question is about using RL to manage a portfolio of financial investments. Portfolio management involves deciding how investments should be allocated and when assets should be bought, sold, or held.

How RL is Used

The agent observes market and portfolio information and selects an investment action. After the market changes, profit, loss, transaction costs, and risk can be used to form the reward. Through repeated interactions, the agent can learn an investment policy.

RL Components

Agent: Portfolio management system.

Environment: Financial market.

State: Asset prices, portfolio value, available cash, holdings, and market conditions.

Action: Buy, sell, hold, or change asset allocation.

Reward: Investment return with penalties for losses, excessive risk, or costs.

Policy: Strategy for selecting investment actions.

Advantages Over Traditional Models

Traditional approaches often use predefined mathematical assumptions, historical statistics, or fixed rules. RL can update its policy as new market data arrives and can include several objectives such as return, risk, and transaction cost.

Risk and Delayed Rewards

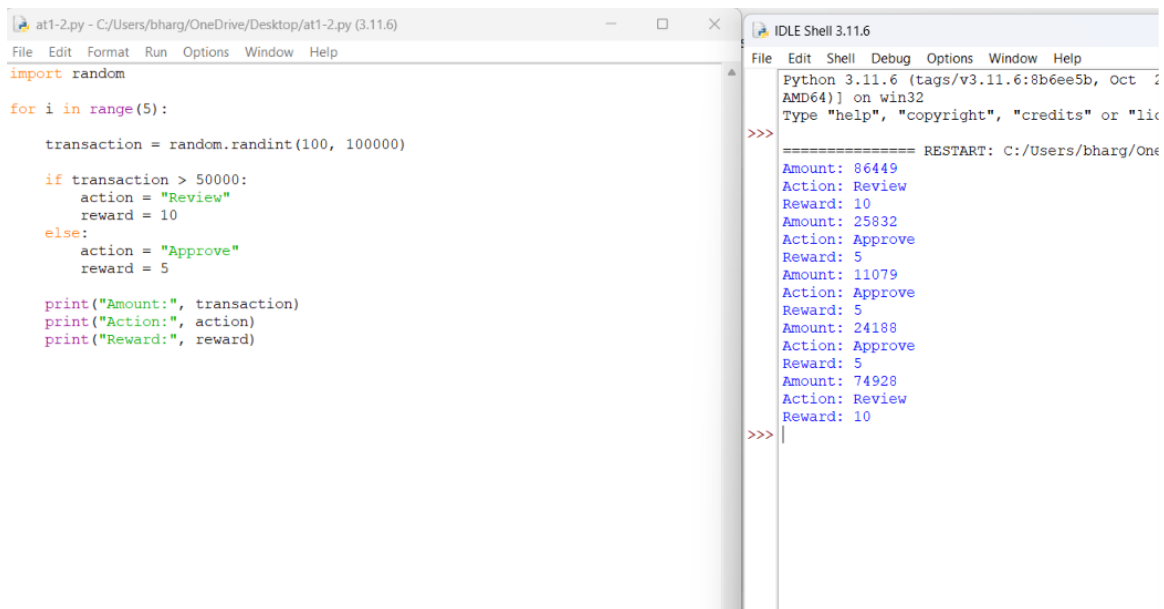
Markets are uncertain, so a decision that appears good can later cause a loss. Risk must therefore be included in the reward or constraints. Rewards can also be delayed because an investment made today may show its real profit or loss days, months, or years later. This makes it harder to determine which earlier decision caused the final result.

Simple Executable Python Code

```
stock_price = 500
later_price = 550

action = "BUY"
reward = later_price - stock_price

print("Current Price:", stock_price)
print("Later Price:", later_price)
print("Action:", action)
print("Reward after price change:", reward)
```

The image shows a screenshot of a Python IDE with two windows. The left window, titled 'at1-2.py - C:/Users/bharg/OneDrive/Desktop/at1-2.py (3.11.6)', contains the following Python code:

```
import random

for i in range(5):

    transaction = random.randint(100, 100000)

    if transaction > 50000:
        action = "Review"
        reward = 10
    else:
        action = "Approve"
        reward = 5

    print("Amount:", transaction)
    print("Action:", action)
    print("Reward:", reward)
```

The right window, titled 'IDLE Shell 3.11.6', shows the output of the script after five iterations:

```
Python 3.11.6 (tags/v3.11.6:8b6ee5b, Oct 2
AMD64) on win32
Type "help", "copyright", "credits" or "lic
>>>
===== RESTART: C:/Users/bharg/One
Amount: 86449
Action: Review
Reward: 10
Amount: 25832
Action: Approve
Reward: 5
Amount: 11079
Action: Approve
Reward: 5
Amount: 24188
Action: Approve
Reward: 5
Amount: 74928
Action: Review
Reward: 10
>>>
```

10. Reinforcement Learning in Renewable Energy Management

About the Question

This question is about using RL to manage renewable energy such as solar and wind power. Renewable generation changes because it depends on environmental conditions.

How RL is Used

The agent observes current energy generation, demand, battery level, weather, and other conditions. It decides whether energy should be used, stored, supplied to the grid, or taken from the battery. Efficient decisions receive positive rewards.

RL Components

Agent: Energy management system.

Environment: Solar panels, wind turbines, batteries, electricity grid, consumers, and weather.

State: Energy generation, demand, battery level, weather, electricity price, and time.

Action: Use energy, store energy, supply energy to the grid, or use battery energy.

Reward: Positive reward for efficient energy use and penalties for shortages or wasted energy.

Policy: Strategy for choosing the best energy-management action.

Environmental Uncertainty

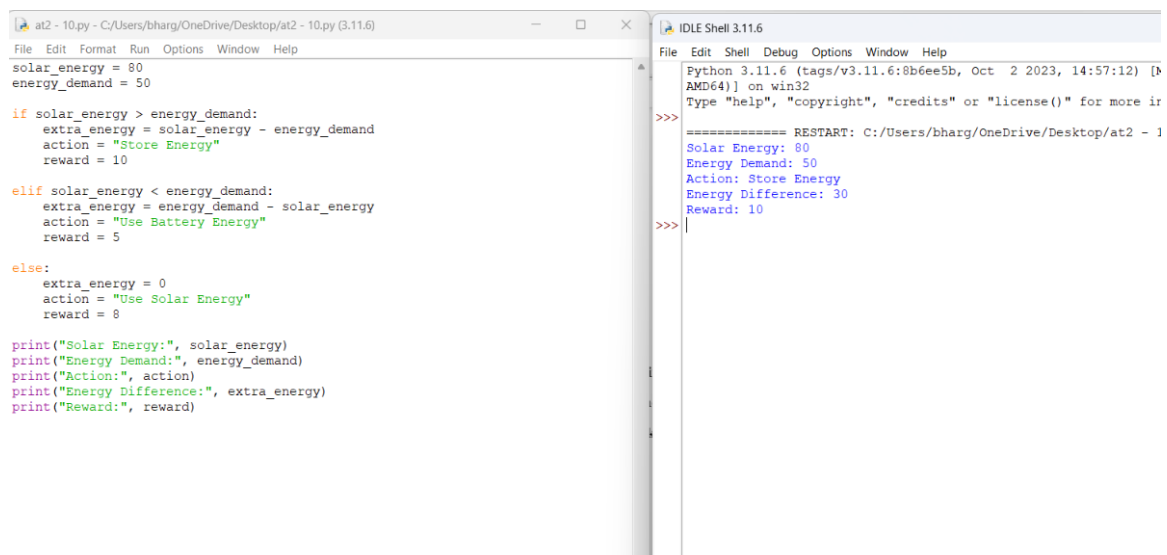
Solar generation depends on sunlight and wind generation depends on wind speed. Weather can change unexpectedly, so the amount of renewable energy available cannot always be predicted accurately. The agent must adapt by using stored battery energy, storing excess energy, or obtaining energy from the grid. Continuous learning helps the system respond better to changing environmental conditions.

Simple Executable Python Code

```
solar_energy = 80
energy_demand = 50

if solar_energy > energy_demand:
    difference = solar_energy - energy_demand
    action = "Store Energy"
    reward = 10
elif solar_energy < energy_demand:
    difference = energy_demand - solar_energy
    action = "Use Battery"
    reward = 5
else:
    difference = 0
    action = "Use Solar Energy"
    reward = 8

print("Solar Energy:", solar_energy)
print("Energy Demand:", energy_demand)
print("Action:", action)
print("Energy Difference:", difference)
print("Reward:", reward)
```



```
at2 - 10.py - C:/Users/bharg/OneDrive/Desktop/at2 - 10.py (3.11.6)
File Edit Format Run Options Window Help
solar_energy = 80
energy_demand = 50

if solar_energy > energy_demand:
    extra_energy = solar_energy - energy_demand
    action = "Store Energy"
    reward = 10

elif solar_energy < energy_demand:
    extra_energy = energy_demand - solar_energy
    action = "Use Battery Energy"
    reward = 5

else:
    extra_energy = 0
    action = "Use Solar Energy"
    reward = 8

print("Solar Energy:", solar_energy)
print("Energy Demand:", energy_demand)
print("Action:", action)
print("Energy Difference:", extra_energy)
print("Reward:", reward)
```

```
IDLE Shell 3.11.6
File Edit Shell Debug Options Window Help
Python 3.11.6 (tags/v3.11.6:8b6ee5b, Oct 2 2023, 14:57:12) [B
AMD64]] on win32
Type "help", "copyright", "credits" or "license()" for more ir
>>>
===== RESTART: C:/Users/bharg/OneDrive/Desktop/at2 - 1
Solar Energy: 80
Energy Demand: 50
Action: Store Energy
Energy Difference: 30
Reward: 10
>>>
|
```